Migration Analysis: MongoDB to PostgreSQL for Genetic Core Layer

Document Version: 1.0

Date: April 2026
Prepared For: Technical Decision Makers
Subject: Feasibility & Recommendation on Consolidating Polyglot Persistence (PostgreSQL + MongoDB) into Single PostgreSQL Database

---

1. Executive Summary

The current system employs a **polyglot persistence** architecture:

- **PostgreSQL (via Drizzle ORM)** – Management layer (users, vendors, patients, orders, samples, shipments, invoices, etc.)
- **MongoDB (via Mongoose ODM)** – Genetic core (variant results, recommendations, patient additional health data, reference pages, etc.)

We have evaluated the technical feasibility, long-term maintenance costs, and business risks of migrating the entire MongoDB genetic layer into PostgreSQL.

Conclusion: Do not migrate. The polyglot design is well-suited to the domain's requirements. Forcing genetic data into a relational schema would introduce significant complexity, performance penalties, and development overhead without tangible benefits. A clear separation of concerns between transactional business data (PostgreSQL) and document-oriented genetic data (MongoDB) is the optimal long-term strategy.

---

2. Current Architecture Overview

Layer	Database	ORM/ODM	Primary Data
Management	PostgreSQL	Drizzle	Users, vendors, patients, orders, samples, shipments, invoices, audit logs
Genetic Core	MongoDB	Mongoose	Variant results (`geneticVariant`, `GeneReportTemp`), recommendations (`*Recommendation` models), patient additional health data (`patientAdditional*`), reference data (`genePageData`, `testMaster`), nutrition data

Data volume estimate (inferred from schema):

- Management: moderate transactional volume (orders, shipments, invoices).
- Genetic: potentially large (hundreds of variants per sample × thousands of samples) – highly read-intensive during report generation.

Access patterns:

- Management: CRUD with complex joins, ACID transactions.
- Genetic: bulk reads of entire documents (e.g., all variants for one sample), occasional writes (result updates, admin overrides).

---

3. Migration Feasibility Analysis

3.1 Technical Feasibility – Yes, but with high effort

Mapping MongoDB's document model to relational tables is possible using techniques such as:

- **Normalization** – splitting nested arrays into separate tables (e.g., `geneticVariant` → 4 tables).
- **EAV (Entity-Attribute-Value)** – for polymorphic `patientAdditional*` models (one generic table with `domain` and `condition_code` columns).
- **JSONB columns** – storing semi-structured data as native JSON in PostgreSQL.

However, each of these approaches has severe drawbacks:

Approach	Drawback
Full normalization	Excessive joins (4+ tables per query), complex insert/update logic, poor read performance for report generation.
EAV	Loses relational integrity, requires pivoting in application code, slower aggregations.
JSONB	Abandons relational benefits for those columns, still requires custom indexing, and you effectively re-implement a document store.

Estimated migration effort:

- Schema design: 2–3 weeks
- ETL script development & testing: 3–4 weeks
- Application code rewrite (Mongoose → Drizzle): 4–6 weeks
- Parallel run & validation: 2–3 weeks
- **Total: 12–16 weeks for a dedicated team of 2–3 developers.**

3.2 Data Integrity & Consistency

MongoDB uses `ObjectId` references; PostgreSQL uses `UUID`. A migration would require:

- Mapping every MongoDB `patientId` (string) to the PostgreSQL `patients.id` (UUID).
- Ensuring all foreign keys are valid after transformation.
- Handling orphaned references and data type mismatches (e.g., `reportTestId` as string vs. `UUID`).

These tasks are error-prone and would require extensive validation.

3.3 Performance Considerations

Current report generation reads a single MongoDB document containing all variant results for a sample – one query, fast.

In a normalized PostgreSQL design, generating the same report might require:

- 1 query to fetch the sample,
- 1 query to fetch the variant groups,
- 1 query per RSID group,
- 1 query per variant detail row.

That could easily become **10–20 SQL queries per report**, severely impacting performance.
Using JSONB would reduce the number of queries but still require parsing JSON in the database or application.

---

4. Pros and Cons of Migration

4.1 Arguments in Favour of Migration

Pro	Validity
Single database – simpler backups, disaster recovery, and monitoring.	True, but modern tools handle multiple databases well (e.g., `pg_dump` + `mongodump`).
Unified ORM – developers only need Drizzle, not Mongoose.	True, but the learning curve for Drizzle is similar to Mongoose; both are well-documented.
Stronger schema enforcement – PostgreSQL constraints prevent malformed data.	True, but Mongoose schemas already provide validation at the application level.
Cross-database joins – e.g., "find all samples with a variant status 'Bad'".	This is the strongest technical argument. However, such joins can be implemented via a lightweight sync table or a reporting service without full migration.

4.2 Arguments Against Migration

Con	Severity
Massive development effort – 12–16 weeks of dedicated work.	High – delays feature development and bug fixes.
High risk of data corruption during ETL.	High – genetic data is critical for patient reports.
Loss of document model flexibility – adding a new condition requires schema changes or EAV workarounds.	Medium – MongoDB allows adding fields instantly.
Performance degradation for report generation due to multiple joins or JSONB parsing.	Medium – could impact user experience.
No clear business ROI – the current system works; migration solves no pressing problem.	High – it's a "nice to have", not a necessity.

---

5. Risk Assessment

Risk	Likelihood	Impact	Mitigation
Data loss or corruption during ETL	Medium	Critical	Extensive testing, backups, dual-write period – still not zero risk.
Performance regression after migration	Medium	High	Load testing and query optimization – but fundamental access pattern mismatch may remain.
Extended downtime during cut-over	Low	High	Can be avoided with blue-green deployment, but complexity is high.
Developer productivity drop (learning new queries)	High	Medium	Training and documentation – acceptable but costly.
Hidden schema inconsistencies (e.g., embedded documents with variant structures)	Medium	Medium	Requires manual inspection of all MongoDB documents.

Overall risk level: HIGH – given the critical nature of genetic data and the lack of urgent need, the risk is unacceptable.

---

6. Final Recommendation

> **Do not migrate the genetic core from MongoDB to PostgreSQL.**

Rationale:

1. **The current polyglot architecture is appropriate** – transactional business data in PostgreSQL, document-oriented genetic data in MongoDB. Each database is used for what it does best.
2. **Migration would be costly and risky** with no tangible business value.
3. **The existing codebase is already well-structured** – Mongoose schemas define clear contracts, and the separation of concerns is clean.
4. **Future maintenance would be harder** – every new genetic condition or report type would require PostgreSQL schema changes, slowing down innovation.

6.1 When Would Migration Be Justified?

- If MongoDB becomes an operational burden (e.g., licensing costs, lack of in-house expertise, performance issues).
- If regulatory compliance explicitly requires a single database (rare).
- If the genetic data volume becomes so large that managing two databases is unsustainable – but this would require millions of samples, at which point a data lake or warehouse would be more appropriate than PostgreSQL.

None of these conditions apply today.

---

7. Alternative Strategy – Optimise the Polyglot Architecture

Instead of migrating, invest in making the current setup more robust and maintainable:

7.1 Define Clear API Boundaries

Create a service layer (e.g., `GeneticDataService`) that encapsulates all MongoDB access. The rest of the application interacts only with this service – never directly with Mongoose. This isolates changes and simplifies future decisions.

7.2 Implement a Lightweight Sync for Cross-Domain Reporting

If you need to query "samples with variant status 'Bad'", maintain a small PostgreSQL table that mirrors key genetic statuses:

```
CREATE TABLE genetic_summary (
  sample_id UUID PRIMARY KEY REFERENCES samples(id),
  variant_statuses JSONB,  -- e.g., {"Lactose": "Bad", "VitaminD": "Good"}
  updated_at TIMESTAMP
);
```

Update this table via a background job or MongoDB change streams. Then you can join without full migration.

7.3 Enhance Monitoring and Backups

- Automate daily backups of both databases.
- Set up monitoring for connection pools, query latency, and disk usage on both systems.
- Use a single cloud provider (e.g., AWS RDS for PostgreSQL + MongoDB Atlas) to simplify operations.

7.4 Document the Data Flow

Create an architecture diagram and data dictionary that clearly shows which data lives where and how they interact. This reduces onboarding time and prevents accidental misuse.

---

8. Conclusion

After thorough analysis of the codebase, data models, access patterns, and migration complexity, **we strongly advise against migrating the genetic core from MongoDB to PostgreSQL**. The current polyglot persistence design is a sound architectural choice for this domain. The effort and risk of consolidation outweigh any theoretical benefits.

Final verdict: **Maintain the polyglot architecture** – focus on operational excellence and clear boundaries between the two databases.

Document prepared by: Technical Architecture Team